

Aqui está o manual revisado, com passos duplicados removidos, didática aprimorada e estrutura mais lógica:

---

# MANUAL DO GITHUB

---

*(Git e GitHub - Guia Prático para Iniciantes)*

## 1. PRIMEIROS PASSOS

---

### Criar um Requirements

---

#### 1. Gerar o arquivo `requirements.txt` automaticamente

Se você já tem um projeto Python com pacotes instalados, execute no terminal:

```
pip freeze > requirements.txt
```

**Isso criará um arquivo `requirements.txt` com todas as bibliotecas instaladas no seu ambiente.**

---

#### 1.1. Criar manualmente (recomendado para projetos organizados)

Se você quer controlar apenas as dependências principais, crie um arquivo `requirements.txt` e liste as bibliotecas com versões específicas (opcional):

```
numpy==1.24.0  
pandas>=1.5.0  
flask  
requests>=2.28.0
```

- `==` → Versão exata

- `>=` → Versão mínima
- Sem versão → Pega a mais recente

---

## 1.2 Instalar dependências a partir do `requirements.txt`

Para instalar todas as dependências em outro ambiente, use:

```
pip install -r requirements.txt
```

---

### Dica extra: Usar `pipreqs` para gerar apenas as dependências usadas no projeto

Se `pip freeze` lista muitos pacotes desnecessários, instale o `pipreqs`:

```
pip install pipreqs
```

Depois execute no diretório do projeto:

```
pipreqs . --force
```

Isso gera um `requirements.txt` apenas com as bibliotecas usadas no código.

## 2.1 Configuração Inicial

No comando `git config --global user.name`, você deve colocar seu nome real (ou como você quer ser identificado nos commits), não seu nome de usuário do GitHub. O email também deve ser o mesmo que você usa no GitHub.

```
# Configurar usuário (global)
git config --global user.name "Fulano"
git config --global user.email "seu-email@exemplo.com" # use o email associ
git config --global init.defaultBranch main
```

Você pode verificar suas configurações atuais com:

```
git config --global --list
```

## 2.2 Primeiro Repositório

### Passo a Passo:

1. Crie um novo repositório no GitHub (com mesmo nome do projeto local)
2. No terminal do vscode, navegue até sua pasta do projeto:

```
cd c:/pasta_do_projeto
```

3. Inicialize o Git:

```
git init
```

4. Conecte ao repositório remoto:

```
git remote add origin https://github.com/seu-usuario/nome-repositorio.git
```

---

## 3. CONTROLE DE ARQUIVOS

### 3.1 O que Subir para o GitHub?

- Arquivos de código, documentação e assets essenciais
- **Evite:** arquivos grandes (>100MB), sensíveis (.env) ou temporários

### 3.2 Usando .gitignore

Crie um arquivo `.gitignore` na raiz com:

```
# Exemplo:  
node_modules/  
.env  
*.log
```

```
*.tmp
```

- Colocar no gitignore os ambientes virtuais, senhas ou outros arquivos sensitive

### Para aplicar:

```
git add .gitignore
git commit -m "Adiciona arquivo .gitignore"
```

---

## 4. PRIMEIRO COMMIT

---

### 4.1 Fluxo Básico

# 1. Adicionar arquivos específicos:

```
git add README.md .gitignore
```

# 2. Preparar commit:

```
git commit -m "Primeiro commit: estrutura inicial"
```

# 3. Enviar para o GitHub:

```
git push -u origin main
```

### 4.2 Atualizações Posteriores

# Adicionar alterações:

```
git add pasta/arquivo.ext
```

# Commitar:

```
git commit -m "Descrição clara das alterações"
```

# Enviar:

```
git push origin main
```

---

## 5. BOAS PRÁTICAS

---

### 5.1 Mensagens de Commit

- Use verbos no imperativo (“Adiciona”, “Corrige”, “Atualiza”)
- Seja específico:
  - ✗ “Mudanças”
  - ✓ “Corrige cálculo de impostos no módulo financeiro”

## 5.2 Estrutura de Pastas

Sugestão para projetos de dados:

```
_projeto/  
├── _1_dados/  
├── _2_scripts/  
├── _3_documentacao/  
└── README.md
```

---

## 6. GIT LFS (PARA ARQUIVOS GRANDES)

---

### 6.1 Configuração

```
# Instalar:  
git lfs install  
  
# Rastrear tipos de arquivo:  
git lfs track "*.csv" "*.zip"  
- aqui você esta rastreando arquivos .csv e .zip  
  
# Verificar:  
git lfs track
```

### 6.2 Uso

```
git add arquivo_grande.csv  
git commit -m "Adiciona dataset inicial"  
git push origin main
```

## 7. SOLUÇÃO DE PROBLEMAS

---

### 7.1 Desfazendo Ações

```
# Modificações não commitadas:  
git restore arquivo.txt  
  
# Remover do staged:  
git restore --staged arquivo.txt  
  
# Alterar último commit:  
git commit --amend -m "Nova mensagem"
```

### 7.2 Reiniciando Repositório

```
rm -rf .git  
git init  
git remote add origin URL  
git add .  
git commit -m "Reinício do projeto"  
git push -f origin main # CUIDADO: apaga histórico remoto!
```

---

## 8. WORKFLOW AVANÇADO

---

### 8.1 Trabalhando com Branches

```
# Criar nova branch:  
git checkout -b nova-feature  
  
# Enviar para o GitHub:  
git push -u origin nova-feature  
  
# Voltar para main:  
git checkout main
```

Aqui está a seção completa sobre **git reset**, organizada de forma didática para iniciantes:

## 9. GIT RESET - CONTROLE DE HISTÓRICO E ARQUIVOS

### 9.1 O que é git reset?

Comando para:

- Desfazer commits
- Remover arquivos da área de staging
- Restaurar o projeto para versões anteriores

⚠ **Cuidado:** Alterações podem ser permanentes. Use com precaução!

---

### 9.2 Tipos de Reset

#### 9.2.1 Reset Soft (--soft)

- **O que faz:** Remove commits mas mantém alterações em staging
- **Quando usar:** Para refazer mensagens de commit ou reorganizar commits pequenos

```
git reset --soft HEAD~1 # Remove o último commit, mantendo alterações prontas p
```

#### 9.2.2 Reset Mixed (padrão)

- **O que faz:** Remove commits e tira arquivos do staging (mas mantém alterações no diretório)
- **Quando usar:** Quando você adicionou arquivos por engano ao staging

```
git reset HEAD~1          # Equivalente a git reset --mixed HEAD~1
git reset arquivo.txt      # Remove arquivo específico do staging
```

#### 9.2.3 Reset Hard (--hard)

- **O que faz:** Apaga completamente commits e alterações não commitadas
- **Quando usar:** Para descartar totalmente alterações indesejadas

```
git reset --hard HEAD~1 # Apaga último commit E todas as alterações não salvas
```

```
git reset --hard abc123 # Volta para o commit específico (hash abc123)
```

### 9.3 Cenários Práticos

#### Cenário 1: Desfazer commit não enviado ao GitHub

```
git reset --soft HEAD~1 # Remove commit mas mantém alterações
# Edite os arquivos se necessário
git commit -m "Nova mensagem correta"
```

#### Cenário 2: Descartar alterações locais não commitadas

```
git reset --hard # CUIDADO: Perde TODAS as alterações não commitadas
```

#### Cenário 3: Corrigir arquivos no staging

```
git add arquivo1.txt arquivo2.txt # Adicionou por engano
git reset arquivo2.txt # Remove apenas arquivo2.txt do staging
```

### 9.4 Reset vs Revert

| Comando    | Altera Histórico?        | Ideal Para                       |
|------------|--------------------------|----------------------------------|
| git reset  | ✔ Sim                    | Correções locais (antes do push) |
| git revert | ✘ Não (cria novo commit) | Correções em commits já públicos |

### 9.5 Boas Práticas

- 1. **Nunca faça reset em commits já enviados ao repositório remoto** (use git revert)
- 2. **Sempre verifique o status antes:**



```
git status
git log --oneline # Veja hashes dos commits
```

### 3. Para salvar alterações temporariamente antes do reset:

```
git stash # Guarda alterações não commitadas
git reset --hard HEAD
git stash pop # Recupera alterações
```

---

## 9.6 Recuperando Commits Apagados

Se você fez reset por engano:

```
git reflog # Mostra todo o histórico de ações
git reset --hard abc123 # Volta para o commit desejado (use hash do reflog)
```

---

### Exemplo Visual

Fluxo de alteração com diferentes tipos de reset:

```
Commit A --- Commit B (HEAD) [Estado atual]
|
├─ soft: mantém alterações em staging
├─ mixed: mantém alterações no dir. de trabalho
└─ hard: remove tudo (volta exatamente para A)
```

Esta seção cobre desde o básico até situações complexas de recuperação, sempre alertando sobre os riscos de cada operação.

---

## OBSERVAÇÕES IMPORTANTES



### Armadilhas Comuns

#### 1. Nunca trabalhe diretamente na branch `main`

- Sempre crie branches para novas funcionalidades ( `git checkout -b nova-branch` )

## 2. Cuidado com `git push -f`

- Sobrescreve o histórico remoto - pode causar perda de trabalho de outros colaboradores

## 3. Problemas com arquivos grandes

- GitHub bloqueia pushes acima de 100MB - use Git LFS ou adicione ao `.gitignore`



## Dicas Valiosas

### 1. Sempre comece com `git status`

- Verifique o estado atual antes de qualquer operação

### 2. Commite frequentemente

- Pequenos commits com mensagens claras > um commit gigante

### 3. Mantenha seu `.gitignore` atualizado

- Adicione padrões de arquivos temporários/desnecessários desde o início

### 4. Convenções de nomenclatura

- Branches: `feat/nome-da-feature`, `fix/correcao-x`
- Tags: `v1.0.0`, `release-2023-10`



## Quando Algo Der Errado

### 1. Use `git log --oneline`

- Visualize o histórico de commits de forma compacta

### 2. Para desfazer quase tudo

bash

Copy

Download

git reflog # Mostra todas as ações realizadas

git reset --hard [hash-do-commit] # Volta para estado anterior

### 3. Conflitos de merge?

- Mantenha a calma! Edite os arquivos conflitantes manualmente, removendo os marcadores <<<<<< , ===== e >>>>>>



## Próximos Passos

### 1. Explore recursos avançados:

- git stash (para salvar trabalho não finalizado)
- git rebase (para limpar histórico de commits)
- GitHub Actions (automatização de workflows)

### 2. Pratique com:

- [GitHub Learning Lab](#)
- [Oh My Git!](#) (jogo educativo)

### 3. Documente seu projeto:

- Crie um [README.md](#) completo
- Adicione licença (MIT, GPL, etc.)

Lembre-se: Todo desenvolvedor já cometeu erros com Git - o importante é aprender com eles!



## OBSERVAÇÕES IMPORTANTES PARTE 2

---

**Modificar o nome da branch principal do seu repositório Git de master para main , siga estes passos:**

## 1. Primeiro, crie a nova branch `main` :

```
git branch -m master main
```

## 2. Faça push da nova branch para o repositório remoto (GitHub):

```
git push -u origin main
```

## 3. Atualize a branch padrão no GitHub:

- Vá até seu repositório no GitHub
- Clique em “Settings” (Configurações)
- No menu à esquerda, clique em “Branches”
- Na seção “Default branch”, clique no ícone de edição (✎) e selecione `main`
- Clique em “Update” (Atualizar)

## 4. Finalmente, delete a branch `master` remota (opcional):

```
git push origin --delete master
```

## 5. Atualize seu repositório local:

```
`` bash
git fetch origin
git remote prune origin
```

Isso fará com que:

- Todos os novos repositórios que você criar usarão `main` como branch padrão (devido à configuração `init.defaultBranch main`)
- Seus repositórios existentes serão migrados de `master` para `main`